SFT Practice

Name: LEI Yulu **ID: 1155244508**

1. data cleaning strategy

Dataset processing is performed locally. A total of 1,000,000 records from Open-Orca/OpenOrca are loaded.
Method：

- Format cleaning: Delete samples with empty question or response.
  - Empty question: 0 records
  - Empty response: 1 record
  - Empty system_prompt: 76,501 records
  - Delete 1 null value, leaving 999,999 records.
- Length filtering: question ≥ 10 characters, response ≥ 20 characters, total length ≤ 4000 characters.
  119,895 invalid records were deleted, leaving 880,104 records.
- Quality Filtering: Removes replies containing rejection messages, clichés, or low-quality reply patterns (14 pattern matching).
  9,131 low-quality records were deleted, leaving 870,973 records.
- Deduplication: Precise deduplication based on the MD5 hash of the question field.
  113,141 duplicate records were removed, leaving 757,832 records.
- Consistency check: Standardize whitespace characters and newline characters.
  The data size remains unchanged after standardization.
- Downsampling: Use cluster-balanced sampling. Clustering is performed using Sentence Embedding + MiniBatchKMeans. Then, sampling is performed from each cluster using a log-balanced strategy to ensure a uniform distribution of task types.
  K = 500, batch_size = 2048, Downsampled to 199,800 data points.

Data value distribution after cleaning:

	Maximum value	Minimum value	Average value
Question length distribution	3973	17	594.2
Response length distribution	3931	20	729.6

The largest cluster has 625 samples, the smallest cluster has 1 sample (non-zero), and the Gini imbalance is 0.9969.
Save the filtered data in JSON format. Then convert it to LlamaFactory's alpha format and register the dataset in LlamaFactory's `dataset_info.json` to adapt it for SFT training.

2. training tuning

Full-parameter supervised fine-tuning of Qwen2.5-0.5B was performed using Google Colab T4. Multiple rounds of optimization were conducted on the parameters in the corresponding hyperparameter configuration .yaml files, as detailed below:

A) After 3 hours of training, I only reached 6% (1211/18750 steps), and it's estimated that it will take more than 47 hours. Based on the Minimind effect, it should normally only take 7-8 hours.

- per_device_train_batch_size: 4, gradient_accumulation_steps: 8, effective batch=32 requires 8 serial forward+backward operations, speed ~8.9s/step.
- bf16: true. T4 (Turing architecture) does not support bfloat16, and the underlying fallback simulation results in a speed loss.
- cutoff_len: 1024, the sequence is too long, resulting in high memory usage.
- preprocessing_num_workers: 8, Colab only supports 2 workers.

This round of modifications:

- bf16: true → fp16: true (T4 natively supports fp16)
- per_device_train_batch_size: 4 → 8
- gradient_accumulation_steps: 8 → 16 (maintaining a valid batch size of 32, but later it was discovered that the direction was incorrect).
- cutoff_len: 1024 → 512
- Added gradient_checkpointing: true (saves approximately 40-60% of activation memory).
- Added optim: adamw_8bit (requires bitsandbytes, saves approximately 2GB of VRAM for optimizer state)
- preprocessing_num_workers: 8 → 2, new dataloader_num_workers: 2
- Add bitsandbytes (adamw_8bit) to the installation dependencies.

B) The second round of analysis revealed the core bottlenecks: missing packing and excessively high gradient accumulation.

- 4.84 seconds per step, 18,750 total steps, estimated 25 hours, still not up to standard.
- With an average OpenOrca sample length of 150-250 tokens, a significant amount of padding is wasted when cutoff_len=512.
  - No packing: Each sample occupies 512 slots exclusively, with a large number of positions filled with PAD tokens
  - Packing: By concatenating multiple short samples to fill 512, the total number of steps is reduced by 2-3x.
- Furthermore, it was discovered that the gradient_accumulation was being performed incorrectly: increasing it from 8 to 16 resulted in progressively slower performance; the number of accumulation steps should be reduced and the batch_size increased.

This round of modifications:

- Added packing: true (reduces total steps)）
- per_device_train_batch_size: 8 → 32(Referencing Minimind's large batch strategy)
- gradient_accumulation_steps: 16 → 4(Reduce the number of serial forwards)
- Added `torch_compile: true` (refer to minimind's `use_compile=1` for PyTorch 2.x graph compilation).
- dataloader_num_workers: 2 → 8 (refer to minimind’s num_workers=8)

Refer to the methods of MiniMind training scripts:

- accumulation_steps=1 (No gradient accumulation, only one forward + backward iteration per step)
- num_workers=8 (Parallel data loading)
- pin_memory=True (Paging memory accelerates CPU→GPU transfer)
- use_compile option (graph compilation optimization using torch.compile)
- scaler is only enabled for float16 (bfloat16 does not require GradScaler)

C) The neat_packing compatibility error, coupled with T4's lack of support for torch_compile and batch=32, resulted in an OutOfMemoryError (OOM).

- `neat_packing` was removed in transformers versions >= 4.53.0. Colab environments update more frequently.
- The speedup effect of `torch_compile` (or `torchdynamo`) only applies to Ampere and later architectures. T4 is a Turing architecture; enabling `torch_compile` not only provides no speedup but also wastes 5-10 minutes of compilation time.
- Qwen2.5 has a very large vocabulary (vocab_size = 151936), which makes the logits tensor exceptionally large. `logits.float()` is called before loss calculation (converting to fp32). Most models have a vocabulary of around 32,000 words; Qwen2.5's 151936 makes the logits memory usage approximately 4.75 times larger.

This round of modifications:

- neat_packing: true → packing: true (downgrades to normal packing, but still reduces the number of steps)
- Remove torch_compile: true (not applicable in T4)

- dataloader_num_workers: 8 → 2 (Reverted; Colab actually only supports 2 workers)
- per_device_train_batch_size: 32 → 8 (reverting to the verified safe value)
- gradient_accumulation_steps: 1 → 4 (Keep valid batch=32)
- cutoff_len: 512 → 384 (Sequence shortened by 25%, computational cost per forward step reduced by approximately 25%)
- num_train_epochs: 2 → 1 (half the number of steps)

Final configuration:

Parameter	**Original value**	**Final value**	**Reasons for the changes**
bf16/fp16	**bf16**	**fp16**	**T4 does not support bf16**
per_device_train_batch_size	**4**	**8**	**Increase throughput**
gradient_accumulation_steps	**8**	**4**	**Reduce the number of serial forward operations**
cutoff_len	**1024**	**384**	**Reduce video memory and computational load**
num_train_epochs	**2**	**1**	**Control total time**
packing	**/**	**true**	**Eliminate padding waste**
gradient_checkpointing	**/**	**true**	**Saves 40-60% of activation value video memory.**
optim	**adamw_torch**	**adamw_8bit**	**Save optimizer video memory ~2GB**
fp16	**/**	**true**	**T4 natively supports fp16**
preprocessing_num_workers	**8**	**2**	**Colab supports upper limits**
dataloader_num_workers	**/**	**2**	**Colab supports upper limits**

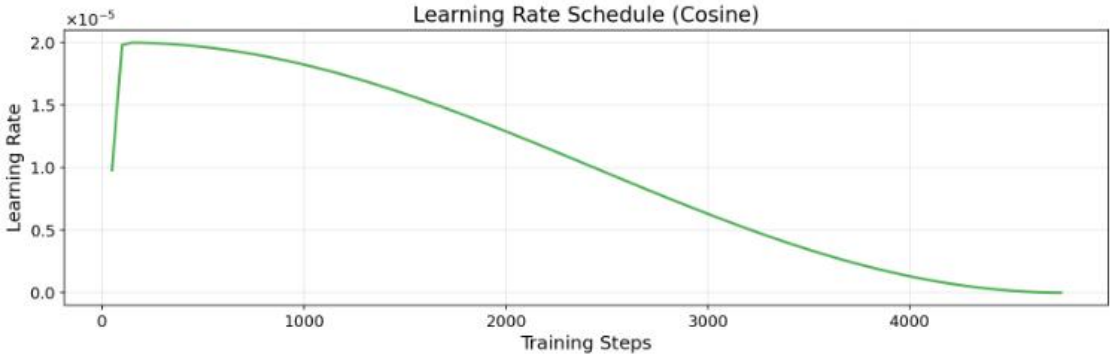
3. evaluation results

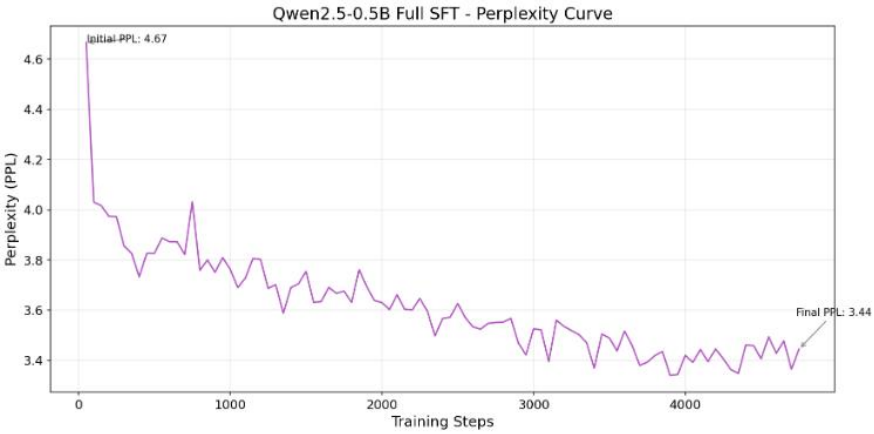
Basic Model vs SFT Model Comparison

Task	Base Acc	SFT Acc	△
mmlu	0.2531	0.4265	+0.1734
arc_easy	0.4924	0.5669	+0.0745
arc_challenge	0.2918	0.3140	+0.0222
hellaswag	0.3837	0.3838	+0.0001
winogrande	0.5533	0.5635	+0.0103
truthfulqa_mc2	0.4263	0.4264	+0.0002
piqa	0.6888	0.6904	+0.0016
boolq	0.6318	0.6838	+0.0520
Average	0.4652	0.5069	+0.0418

4. conclusions & analysis

Generate training-related charts:





- Total training steps: Approximately 4,600 steps（packing +max_samples=200,000 + cutoff_len=384）
- Training duration: Approximately 5.7 hours
- GPU device: Google Colab T4 (16GB VRAM, Turing architecture)

Target	Initial value	Final value	Change
Training loss	1.5405	1.2369	↓ 19.7%
Perplexity(PPL)	4.67	3.44	↓ 26.3%

Significant results were achieved in full parameter fine-tuning on Qwen2.5-0.5B: the training loss decreased from 1.5405 to 1.2369 (a reduction of 19.7%), the perplexity decreased from 4.67 to 3.44 (a reduction of 26.3%), and the curves converged smoothly, indicating that the model effectively learned the conditional distribution of command-response and improved prediction confidence.

Positive gains were achieved across all eight evaluation benchmarks, with the average accuracy improving from 46.52% to 50.69% (+4.18 percentage points). The improvement was particularly significant in instruction comprehension tasks: MMLU +17.34, BoolQ +5.20, and ARC-Easy +7.45, validating that full parameter fine-tuning outperforms PEFT methods (such as LoRA) on small models.

The OpenOrca dataset covers a wide range of instruction types and is highly compatible with the distribution of knowledge-answering tasks such as MMLU and BoolQ, which is a key reason for the significant improvements in these benchmarks. However, the improvements in tasks requiring deep reasoning or fact-checking (HellaSwag, TruthfulQA MC2) are minimal (<0.01 pp), indicating that simply aligning instruction formats is insufficient to improve such capabilities, and more relevant training data coverage is needed.